Ne rien inscrire dans ce cadre

Prénom .....

Nom .....

Promotion .....

Groupe .....

Promotions : L2, L2-BN, L2-BDX, L2-P

Module : Programmation Orientée Objet en Java

02/07/2024 de 15H30 à 16H30

Examen sur table

Sujet rédigé par : Madame Cherifa Ben Khelil

Supports autorisés :

Calculatrice autorisée : **non**

Documents autorisés : **non**

Traducteur électronique : **non**

Dictionnaire : **non**

Consigne :

Merci de restituer : **le sujet avec vos réponses**

Rappel :

- Tous les appareils électroniques (téléphones portables, ordinateurs, tablettes, montres connectées, accès à internet …) doivent être éteints et rangés.
- Il est interdit de communiquer.
- Toute fraude ou tentative de fraude fera l'objet d'un rapport de la part du surveillant et sera sanctionnée par la note zéro, assortie d'une convocation devant le conseil de discipline. Aucune contestation ne sera possible. Tous les documents et supports utilisés frauduleusement devront être remis au surveillant.
- Aucune sortie de la salle d'examen ne sera autorisée avant la moitié de la durée de l'épreuve.

Ne rien inscrire dans ce cadre

S'il vous plaît, n'écrivez pas au crayon !!

Partie I : Compréhension (6 points)

Cochez **la** bonne réponse :

Question 1 : Quelle est la différence entre une classe et un objet en Java ?
☐ Une classe est un modèle pour créer des objets, tandis qu'un objet est une instance spécifique de la classe. ☐ Une classe est un objet créé à partir d'un modèle, tandis qu'un objet est un modèle pour créer des classes. ☐ Une classe est une variable globale, tandis qu'un objet est une variable locale. ☐ Une classe est une méthode statique, tandis qu'un objet est une méthode non statique.
Question 2 : Comment créer une instance d'une classe en Java ?
☐ En utilisant le mot-clé `instanceof` suivi du nom de la classe. ☐ En utilisant le mot-clé `this` suivi du nom de la classe. ☐ En utilisant le mot-clé `new` suivi du nom de la classe. ☐ En utilisant le mot-clé `extends` suivi du nom de la classe.
Question 3 : Quelle est la différence entre une méthode statique et une méthode non statique ?
☐ Une méthode statique appartient à une instance spécifique de la classe, tandis qu'une méthode non statique est partagée par toutes les instances de la classe. ☐ Une méthode statique peut être appelée sans créer d'objet, tandis qu'une méthode non statique nécessite une instance de la classe.

☐ Une méthode statique peut accéder aux variables d'instance, tandis qu'une méthode non statique ne le peut pas.

☐ Une méthode statique est définie dans l'interface, tandis qu'une méthode non statique est définie dans la classe.

Question 4 : Lequel de ces mots clés est utilisé pour faire référence à l'instance courante lors d'un appel de méthode ?

☐ super

☐ new

☐ this

☐ reference

☐ Aucune de ces réponses n'est vraie.

Question 5 : Comment définir une constante publique de classe de type entier, nommée LONGUEUR de valeur 5, en tant qu'instance ?

☐ public static const int LONGUEUR = 5;

☐ public const int LONGUEUR = 5;

☐ public static final int LONGUEUR = 5;

☐ public final int LONGUEUR = 5;

Question 6 : Quelle est la différence entre une interface et une classe abstraite ?

☐ Une interface peut contenir des méthodes implémentées, tandis qu'une classe abstraite ne le peut pas.

☐ Une interface peut avoir des constructeurs, tandis qu'une classe abstraite ne le peut pas.

☐ Une classe abstraite peut contenir des méthodes implémentées, tandis qu'une interface ne le peut pas.

☐ Une interface peut hériter d'une classe abstraite.

Question 7 : Quelle est la signification du mot-clé ***super*** en Java ?

☐ Il permet de déclarer une variable globale.

☐ Il permet d'appeler la méthode parente.

☐ Il permet de créer une nouvelle instance d'une classe.

☐ Il permet de définir une méthode abstraite.

Question 8 : Laquelle de ces méthodes de la classe ***Object*** peut être utilisée pour retourner une chaîne de caractères ?

☐ string

☐ toString

☐ ToString

☐ tostring

☐ display

Question 9 : Quelle est la différence entre ***public***, ***private*** et la visibilité par ***défaut*** en Java ?

☐ ***public*** permet l'accès depuis n'importe où, ***private*** limite l'accès à la classe elle-même et par défaut permet l'accès aux classes du même package.

☐ ***public*** permet l'accès depuis n'importe où, ***private*** limite l'accès à la classe elle-même et par défaut permet l'accès depuis n'importe où.

☐ ***public*** permet l'accès aux classes du même package, ***private*** limite l'accès à la classe elle-même et par défaut permet l'accès aux classes du même package.

☐ ***public*** permet l'accès depuis n'importe où, ***private*** limite l'accès à la classe elle-même et par défaut n'existe pas en java.

Question 10 : Quelle est la sortie du code suivant ?

```
class Example1 {
   public static void main(String[] args) {
       int x = 5;
       if (x < 10) {
           System.out.println("x est supérieur à 10");
       } else {
           System.out.println("x est inférieur à 10");
}}}
```

☐ Erreur d'exécution/compilation

☐ x est supérieur à 10

☐ x est inférieur à 10

☐ x est supérieur à 10 ensuite x est inférieur à 10

☐ aucune sortie

Question 11 : Quelle est la sortie du code suivant ?

```
public class Example2 {
   public static void main(String[] args) {
       int x = 5;
       while (x != 0) {
            System.out.println(x);
            x-=2; }}}
```

☐ Erreur d'exécution/compilation

☐ 0

☐ -2

☐ 5

3

1

☐ boucle infinie

Question 12 : Quelle est la sortie du code suivant ?

```
public class Example3 {
    public static void main(String[] args) {
        String str1 = new String("hello");
        String str2 = new String("hello");
        String str3=str2;
        System.out.print(str1.equals(str2)+" ");
        System.out.print((str1 == str2)+" ");
        System.out.print(str1.equals(str3)+" ");
        System.out.print(str1 == str3);
    }}
```

☐ false false false false

☐ true true true true

☐ true true false false

☐ false true false true

☐ false true false false

☐ true false false true

☐ true false true false

Partie II : Codage (14 points)

Le tic-tac-toe[1], également connu sous le nom de « morpion » (par analogie avec le jeu de morpion) et « oxo » en Belgique, est un jeu de réflexion pour deux joueurs se jouant à tour de rôle. L'objectif est d'être le premier à aligner trois de ses symboles.

Le jeu se joue en dessinant sur du papier avec un crayon. Deux joueurs s'affrontent en remplissant chacun à leur tour une case de la grille avec le symbole qui leur est attribué : O ou X. Le gagnant est celui qui parvient à aligner trois symboles identiques horizontalement, verticalement ou en diagonale. Traditionnellement, le joueur utilisant les X commence la partie.

[1] Source wikipedia https://fr.wikipedia.org/wiki/Tic-tac-toe

Exemple de partie gagnée par le joueur X :

Exemple de partie nulle :

Nous avons défini une hiérarchie pour implémenter le jeu de Tic-Tac-Toe en Java sous forme de classes :

1. **Game** : Classe de base pour tous les jeux. Cette classe contient des méthodes communes comme *switchPlayer* et *getCurrentPlayer*.
2. **BoardGame** : Classe de base pour les jeux de plateau. Cette classe hérite de **Game** et ajoute des fonctionnalités spécifiques aux jeux de plateau, comme l'initialisation et l'affichage du plateau, la vérification des mouvements valides, et la vérification si le plateau est plein.
3. **TicTacToe** : Classe spécifique pour le jeu de Tic-Tac-Toe. Hérite de **BoardGame** et implémente le jeu spécifique de Tic-Tac-Toe, y compris la logique pour déterminer si un joueur a gagné.

Travail demandé: Complétez le code de ces trois classes afin d'exécuter correctement les scénarios suivants:

1. Le programme commence son exécution en affichant le plateau de jeu du Tic-Tac-Toe avec une grille vide initiale (tableau 2D de 3 lignes et 3 colonnes de caractères '-'), suivi du message : "*Joueur X, veuillez entrer la ligne (0-2) et la colonne (0-2) :*", indiquant ainsi où il souhaite placer son symbole (X).

```
- - -
- - -
- - -
Joueur X, entrez la ligne (0-2) et la colonne (0-2) :
1
1
- - -
- X -
- - -
Joueur O, entrez la ligne (0-2) et la colonne (0-2) :
```

1.1. Dans cet exemple d'exécution, le joueur X choisit d'entrer 1 pour la ligne et 1 pour la colonne. Le symbole (X) est alors placé au centre de la grille, qui est ensuite affichée avec le symbole placé. Ensuite, ce sera au tour du joueur O.

1.2. Si le joueur entre des valeurs invalides, le message "*Mouvement invalide. Réessayez.*" s'affichera et le joueur devra saisir à nouveau ses choix.

```
Joueur X, entrez la ligne (0-2) et la colonne (0-2) :
4
5
Mouvement invalide. Réessayez.
```

2. Le jeu se poursuit jusqu'à ce qu'un joueur gagne, auquel cas le résultat est affiché, ou que la grille soit remplie sans vainqueur, ce qui entraîne l'affichage de "Match nul !".

```
X - -
O X O
- - X
Joueur X a gagné !
```

```
X O O
O X X
X X O
Match nul !
```

1. Complétion de la classe Game (2 points):

a. Complétez la méthode ***switchPlayer*** qui modifie le joueur actuel de 'X' à 'O' ou de 'O' à 'X'. **1 point**

b. Implémentez la méthode ***getCurrentPlayer*** qui renvoie le joueur actuel. **1 point**

```
public abstract class Game {
    protected char currentPlayer;

    // Initialise le joueur actuel avec le joueur donné
    public Game(char firstPlayer) {
        this.currentPlayer = firstPlayer;
    }

    // Méthode abstraite que les sous-classes doivent implémenter pour définir
le déroulement du jeu
    public abstract void play();

    // Complétez la méthode switchPlayer
    protected void switchPlayer() {

        ..............................................................................

        ..............................................................................

        ..............................................................................

        ..............................................................................
```

```
        ..............................................................................
        ..............................................................................
        ..............................................................................
        ..............................................................................
        ..............................................................................
    }
    // Implémentez la méthode getCurrentPlayer
    public..............................................................................{
        ..............................................................................
        ..............................................................................
    }
}
```

2. **Complétion de la classe BoardGame (6 points):**
   - ***a.*** Complétez le constructeur de la classe ***BoardGame*** pour initialiser correctement le joueur actuel et le plateau de jeu (le tableau 2D ***board***). **1 point**
   - ***b.*** Complétez la méthode ***initializeBoard*** pour qu'elle initialise un plateau vide en remplissant toutes les cases avec le caractère '-'. **1 point**
   - ***c.*** Complétez la méthode ***printBoard*** qui permet d'afficher le plateau de jeu sous forme de grille (voir exemple d'exécution plus haut). **1 point**
   - ***d.*** Complétez la méthode ***isValidMove*** qui prend en paramètre le numéro de ligne et le numéro de colonne du mouvement. Cette méthode doit vérifier si le mouvement est valide, c'est-à-dire s'il s'agit d'une case vide et si les indices sont dans les limites du plateau. Elle doit retourner true si le mouvement est valide, et false sinon. **1,5 points**
   - ***e.*** Complétez la méthode ***isBoardFull*** qui vérifie si le plateau est entièrement rempli (retourne true) ou s'il reste au moins une case vide (retourne false). **1,5 points**

```
public abstract class BoardGame extends Game {
    protected char[][] board;
    // Complétez le constructeur
    public BoardGame(char firstPlayer) {
        ..............................................................................
        board = new char[3][3];
        ..............................................................................
```

```
    ………………………………………………………………………………………………………………………………………………………………
}
// Complétez la méthode initializeBoard
protected void initializeBoard() {
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
}
// Complétez la méthode printBoard
protected void printBoard() {
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
    ………………………………………………………………………………………………………………………………………………………………
```

```
    ……
}

// Complétez la méthode isValidMove
protected boolean isValidMove(int row, int col) {
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
    ……
}
// Complétez la méthode isBoardFull
protected boolean isBoardFull() {
    ……
    ……
    ……
```

```
        ……
        ……
        ……
        ……
        ……
        ……
        ……
        ……
        ……
        ……
        ……
        ……
        ……
        ……
    }
    // Méthode abstraite pour vérifier si un joueur a gagné
    protected abstract boolean hasWon(char player);
}
```

3. Complétion de la classe TicTacToe (6 points) :

a. Définissez le ***constructeur*** de la classe TicTacToe. **0,5 point**

b. Complétez le bloc d'instructions manquant dans la méthode ***play*** qui gère le déroulement du jeu en se basant sur les indications suivantes **(3 points)**:

b.1. Placer le symbole du joueur actuel sur la case spécifiée.

b.2. Vérifie si le joueur actuel a gagné après son coup :

- Affiche le plateau mis à jour
- Annonce la victoire de ce joueur (exemple : Joueur X a gagné !)
- Utilise break; pour terminer la partie.

b.3. Si le joueur actuel n'a pas gagné après son coup :

- Vérifie si le plateau est plein sans vainqueur après le coup.
- Affiche le plateau mis à jour

- Affiche le message ‘Match nul !’.
- Utilise break; pour terminer la partie.

b.4. Passe au joueur suivant pour le prochain tour.

c. Complétez la méthode ***hasWon*** pour vérifier si le joueur a gagné en examinant toutes les lignes, colonnes et diagonales du plateau. **1,5 points**

d. Complétez la méthode main afin de lancer correctement une partie de jeu. **1 point**

```
import java.util.Scanner;

public class TicTacToe extends BoardGame {

   // Définissez le constructeur
    public TicTacToe() {

        ..............................................................................................................

        ..............................................................................................................

    }

    // Complétez la méthode play
    @Override
    public void play() {
        Scanner scanner = new Scanner(System.in);

        while (true) {
            printBoard();// Affiche l'état actuel du plateau
            System.out.println("Joueur " + currentPlayer + ", entrez la ligne
(0-2) et la colonne (0-2) :");
            int row = scanner.nextInt();// Lecture de la ligne saisie par le joueur
            int col = scanner.nextInt();// Lecture de la colonne saisie par le joueur

            if (isValidMove(row, col)) {

            //à compléter, voir les indications 3.b

        ..............................................................................................................

        ..............................................................................................................

        ..............................................................................................................

        ..............................................................................................................

        ..............................................................................................................

        ..............................................................................................................

        ..............................................................................................................

        ..............................................................................................................
```

……

……

……

……

……

……

……

……

……

……

……

……

……

```
            } else {
                System.out.println("Mouvement invalide. Réessayez.");
            }
        }

        scanner.close();
    }

    // Complétez la méthode hasWon
    @Override

    protected boolean hasWon(char player) {

    // Gagné sur une ligne

    for (int i = 0; i < 3; i++) {

        if (…… == player && …… == player &&

…… == player) {

                return true; }}

    // Gagné sur une colonne

    for (int j = 0; j < 3; j++) {

        if (……== player && ……== player &&
```

```
…… == player) {
                return true; }}
     // Gagné sur l'une des diagonales
     if ((…… == player && ……== player &&
…… == player) || if (……== player &&
……== player && …… == player)){
            return true; }
        return false;
    }
       // Complétez la méthode main
    public static void main(String[] args) {
        TicTacToe game = ……
        game……
    }
}
```